

Bayesian Optimization with Ax: Student Guide

Readable practical guide for chemistry experiments with categorical and mixture parameters.

Ask Ax for trial (suggest conditions) Run experiment (measure outcomes) Tell Ax result (update model) Repeat until stopping criterion

■ Tip: Read Section 6 and Section 14 first if you are designing your first campaign.

Bayesian Optimization with Ax: A Student's Guide

This guide explains how to use the **Ax platform** for Bayesian optimization (BO) in the context of chemical reaction experiments — specifically for problems involving categorical inputs like solvent choice and acid type, combined with continuous inputs like acid concentration.

A ready-to-run reference implementation for your exact problem lives in [recommenders/degradation_optimizer.py](#).

1. What Is Bayesian Optimization?

When you run a chemistry experiment, every trial costs time and materials. You want to find the best conditions (maximize yield, minimize side reactions, use mild acids) without exhaustively testing every combination.

Bayesian optimization (BO) does this by:

1. **Building a surrogate model** — a probabilistic model (usually a Gaussian Process, or GP) that predicts how your output will look across the full parameter space, based on experiments you've already done. It also tells you *how uncertain* it is in each region.
2. **Using an acquisition function** — a mathematical rule that balances "try places where we think it'll be good" (exploitation) against "try places we're still uncertain about" (exploration), to decide what to run next.
3. **Repeating**: run the trial, feed back the result, update the model, repeat.

With ~5–30 trials you can often find near-optimal conditions that would take hundreds of random experiments to find by grid search.

2. API Version Note

This codebase uses `ax-platform==1.1.x` with the `AxClient` interface. Meta has released a newer API (version 1.2.x) that uses a `Client` class with a somewhat different syntax.

Feature	AxClient (1.1.x, used here)	Client (1.2.x, newer)
Import	<code>from ax.service.ax_client import AxClient</code>	<code>from ax.api.client import Client</code>
Create experiment	<code>ax_client.create_experiment(parameters=parameters)</code>	<code>client.configure_experiment(parameters=parameters)</code>
Objectives	<code>objectives={name: ObjectiveProperties(...)}</code>	<code>client.configure_optimization(objectives=objectives)</code>
Get suggestions	<code>ax_client.get_next_trials(max_trials=n)</code>	<code>client.get_next_trials(max_trials=n)</code>
Report results	<code>ax_client.complete_trial(trial_index=trial_index, raw_data=raw_data)</code>	<code>client.complete_trial(trial_index=trial_index, raw_data=raw_data)</code>
Best result	<code>ax_client.get_pareto_optimal_parameters()</code>	<code>client.get_pareto_frontier()</code>

Use the `AxClient` API when working with this codebase. The `Client` API exists and is documented at [ax.dev](#), but it is not compatible with the installed version.

3. The Basic Loop

```
from ax.service.ax_client import AxClient, ObjectiveProperties
ax_client = AxClient()
ax_client.create_experiment( parameters=[...], # defined below
                             objectives={...}, # defined below )
for round in range(num_rounds):
    suggestions = ax_client.get_next_trials(max_trials=batch_size)
    for trial_index, params in suggestions.items():
        result = run_my_experiment(params) # your lab/code here
    ax_client.complete_trial(trial_index=trial_index, raw_data=result)
    best = ax_client.get_pareto_optimal_parameters() # multi-objective # or
            ax_client.get_best_parameters() # single objective
```

4. Defining Parameters

Parameters describe the space Ax is allowed to search. Each parameter has a name, a type, and type-specific fields.

4.1 Range parameters (continuous or integer)

Use for anything with a meaningful numeric scale — concentrations, temperatures, times.

```
{ "name": "acid_molar_excess", "type": "range", "bounds": [1.0, 5.0], # [min, max]
  "value_type": "float", # or "int" for discrete integers "log_scale": False, # set True if
  the effect is multiplicative (e.g. pH) }
```

4.2 Choice parameters (categorical)

Use for discrete, unordered options like solvent identity or acid type.

```
{ "name": "solvent", "type": "choice", "values": ["toluene", "2-MeTHF", "EtOAc", "DCM"],
  "is_ordered": False, # True if values have a natural order (e.g. Low/Med/High)
  "sort_values": False, # Ax internal ordering }
```

```
{ "name": "acid", "type": "choice", "values": ["TFA", "HCl", "H2SO4"], "is_ordered":
  False, }
```

Note on how Ax handles categorical parameters internally: Ax converts categorical choice parameters into a numeric representation using one-hot encoding before fitting the GP. This works correctly but has a cost: with n choices you add n input dimensions, and GPs scale poorly to high dimensions. With 3–5 solvents and 2–3 acids this is fine. See Section 6 for alternatives if your categorical space grows.

4.3 Fixed parameters (not optimized)

If a parameter should be the same every trial (e.g. reaction time while you're not optimizing it yet), declare it fixed:

```
{ "name": "reaction_time_h", "type": "fixed", "value": 24.0, }
```

5. Defining Objectives

5.1 Single objective

```
ax_client.create_experiment( parameters=[...], objective_name="degradation_ratio",
  minimize=False, # maximize )
```

5.2 Multi-objective (your case)

```
from ax.service.ax_client import AxClient, ObjectiveProperties
ax_client.create_experiment( parameters=[...], objectives={ "max_degradation_ratio":
ObjectiveProperties(minimize=False, threshold=2.0), "degradation_rate_constant":
ObjectiveProperties(minimize=False, threshold=-0.5), "acid_molar_excess":
ObjectiveProperties(minimize=True, threshold=5.0), }, )
```

`threshold` is a **reference point** — it defines a baseline below which a result is considered not worth considering. Setting it correctly matters: too lenient and the Pareto frontier becomes cluttered; too strict and no trials count as feasible during early exploration. A good starting point is a slightly-worse-than-expected result.

After optimization, retrieve the Pareto frontier — the set of trials where no objective can be improved without worsening another:

```
pareto = ax_client.get_pareto_optimal_parameters() # Returns: {trial_index: (parameters,
(means, covariances))}
```

6. Categorical Parameters: To Featurize or Not?

This is the most important design decision for your specific problem. You have two architectural options.

Option A: Treat solvents and acids as raw categories (recommended for your case)

```
{"name": "solvent", "type": "choice", "values": ["toluene", "2-MeTHF", "EtOAc"],
"is_ordered": False}
```

How it works: Ax one-hot encodes the categories before fitting the GP. The model learns from each individual solvent's experimental results.

Pros: - Simple; no domain knowledge required. - Works correctly out of the box. - Each solvent is treated as a distinct identity — no spurious distance assumptions.

Cons: - Cannot predict the behavior of a *new* solvent it has never seen. - Sobol initialization will sample all combinations, which can mean you need more initial trials (rule of thumb: $\sim 2\times$ the number of categories as initial trials). - With many categories ($\sim 10+$), the one-hot space gets large and GP performance degrades.

Recommended when: you have 3–6 solvents and plan to run all of them at least once. This is the approach in `degradation_optimizer.py` and is appropriate for your problem.

Option B: Featurize solvents/acids as numerical descriptors

Instead of "toluene", you represent it as a vector of physicochemical properties:

Solvent	Dielectric constant (ϵ)	Polarity index	Boiling point (°C)	Hansen Hd
Toluene	2.38	2.4	111	18.0
2-MeTHF	6.97	4.0	80	16.9
EtOAc	6.02	4.4	77	15.8

Then `solvent` becomes several range parameters:

```
{"name": "solvent_epsilon", "type": "range", "bounds": [2.0, 8.0], "value_type": "float"}, {"name": "solvent_polarity", "type": "range", "bounds": [2.0, 5.5], "value_type": "float"},
```

Pros: - Can potentially generalize to new, unseen solvents. - Works better in high-dimensional categorical spaces (10+ solvents). - May converge faster if the true mechanism is driven by a particular property (e.g. ϵ).

Cons: - Requires you to choose and justify which descriptors matter. - The GP now optimizes over a continuous solvent space — you'll get suggestions for " $\epsilon = 4.3$ " that don't correspond to any real solvent. You must either round to the nearest available solvent, or accept that the model is hypothetical. - Descriptor choice introduces bias: if the key property isn't in your feature vector, the model will fail silently.

Recommended when: you have many solvents (8+), strong prior knowledge of which physical properties are relevant, or you explicitly want to generalize across a solvent library.

Option C: Hierarchical / mixed approach

Maintain a choice parameter for solvent *identity*, but also add derived numeric descriptors as fixed parameters that you update per trial. This is complex and not directly supported as a first-class concept in `AxCliEnt`. Not recommended unless you have a specific reason.

Option D: Composition-vector parameterization (mixtures that sum to 1)

If you want to allow **mixtures** of solvents (and/or acids), define each component as its own range parameter in `[0, 1]`, then enforce a sum constraint.

Example for a 3-solvent mixture:

```
parameters=[ {"name": "x_toluene", "type": "range", "bounds": [0.0, 1.0], "value_type":  
"float"}, {"name": "x_2MeTHF", "type": "range", "bounds": [0.0, 1.0], "value_type":  
"float"}, {"name": "x_EtOAc", "type": "range", "bounds": [0.0, 1.0], "value_type":  
"float"}, ] parameter_constraints=[ "x_toluene + x_2MeTHF + x_EtOAc <= 1.0", "x_toluene +  
x_2MeTHF + x_EtOAc >= 1.0", ]
```

Using both ≤ 1.0 and ≥ 1.0 gives an equality constraint ($\text{sum} = 1$) with AxClient's linear-inequality interface.

For acids, do the same:

```
"x_TFA + x_HCl + x_H2SO4 <= 1.0", "x_TFA + x_HCl + x_H2SO4 >= 1.0",
```

Practical notes: - This is a good option when blending is physically meaningful and interactions between components are expected to matter. - It increases effective dimensionality quickly. If you model both solvent and acid mixtures, increase Sobol initialization budget. - A common simplification is to drop one component and recover it as $x_{\text{last}} = 1 - \text{sum}(\text{other_components})$. That reduces dimension and can improve sample efficiency. - Keep lower bounds at 0.0 to preserve physical feasibility. - If you need sparse blends (e.g. at most 2 solvents nonzero), that is a combinatorial constraint and is not directly expressible as a simple linear Ax parameter constraint.

For your current scale (about 3-5 solvents and 2-3 acids), a practical decision rule is: - Use categorical identity parameters if each experiment uses one solvent and one acid, and mixtures are not part of the chemistry question. - Use composition vectors only if you genuinely expect blend effects to change outcomes (synergy, antagonism, or selectivity shifts) enough to justify the extra trials.

In other words: your instinct is right. The main reason to adopt composition vectors is that mixtures might matter scientifically, not because they are simpler for the optimizer.

7. Parameter Constraints

7.1 Parameter constraints (linear relationships between inputs)

These prevent the optimizer from suggesting physically impossible or impractical combinations. Written as linear inequalities.

```
# Example: total solvent fraction must not exceed 1.0 # (if you have two solvent  
fractions v1 + v2 <= 1.0) ax_client.create_experiment( parameters=[...],  
parameter_constraints=["v1 + v2 <= 1.0"], objectives={...}, )
```

Constraints use the parameter names directly and support `<=` and `>=`. Only linear constraints are supported in `AxClient`. Nonlinear constraints (e.g. `v1 * v2 <= 0.1`) are not supported and must be encoded differently or enforced post-hoc.

This codebase uses parameter constraints in `calibration_modular_v2/bayesian_recommender.py` to enforce physical volume limits across pipetting parameters (e.g. `overaspirate_vol + post_asp_air_vol <= tip_volume`).

7.2 Outcome constraints (thresholds on experimental results)

These tell Ax to *only accept* solutions that meet a minimum standard on a secondary metric — the optimizer is not penalized for *exceeding* the threshold, only for falling below it.

In `AxClient`, outcome constraints are implemented by setting `threshold` in `ObjectiveProperties`, or by adding a metric with `minimize=True` (or `False`) and a `threshold`:

```
objectives={ "yield": ObjectiveProperties(minimize=False, threshold=0.50), # must be >=
50% "impurity_pct": ObjectiveProperties(minimize=True, threshold=0.10), # must be <= 10%
}
```

In practice, the `threshold` for a minimize objective is the *upper* acceptable limit, and for a maximize objective it is the *lower* acceptable limit. Points that violate all thresholds do not contribute to the Pareto frontier.

8. Seeding the Optimizer

8.1 Sobol initialization (built-in)

By default, `AxClient` starts with a **Sobol** quasi-random sequence to spread initial trials evenly before switching to Bayesian. You control this via a custom `GenerationStrategy`:

```
from ax.modelbridge.generation_strategy import GenerationStrategy, GenerationStep from
ax.modelbridge.factory import Models gs = GenerationStrategy(steps=[ GenerationStep(
model=Models.SOBOL, num_trials=8, # run 8 quasi-random trials first
min_trials_observed=8, # wait for all results before switching max_parallelism=8,
model_kwargs={"seed": 42}, # fix seed for reproducibility ), GenerationStep(
model=Models.BOTORCH_MODULAR, num_trials=-1, # run indefinitely max_parallelism=1,
model_gen_kwargs={"botorch_acqf_class": qNoisyExpectedHypervolumeImprovement}, ), ])
ax_client = AxClient(generation_strategy=gs)
```

How many Sobol trials? A rule of thumb for GPs is: $\sim 2 \times (\text{number of parameters})$ for a good initial space coverage. - Add ~ 1 extra per categorical level beyond 2 (e.g. 4 solvents adds ~ 2 extra Sobol

trials). - Minimum is 5–6 regardless of dimensionality.

For your problem (1 continuous + 1 categorical acid with 2–3 levels + 1 categorical solvent with 3–5 levels), **8–12 Sobol trials** is a reasonable starting point.

8.2 Warm-starting from previous experiments

If you have already run some experiments manually (or in a pilot study), you can inject them into the model *before* asking for suggestions. This avoids wasting Sobol budget on regions you've already explored.

```
# Attach a previous trial by specifying both parameters and outcomes _, trial_index =
ax_client.attach_trial( parameters={"acid": "TFA", "solvent": "toluene",
"acid_molar_excess": 2.5} ) ax_client.complete_trial( trial_index=trial_index, raw_data={
"max_degradation_ratio": (3.1, 0.0), # (value, SEM) "degradation_rate_constant": (-0.3,
0.0), "acid_molar_excess": (2.5, 0.0), }, )
```

The (value, SEM) tuple allows you to pass measurement noise. If your measurement is essentially deterministic (e.g. plate-reader absorbance), use SEM=0.0. If you have replicate measurements, compute the standard error of the mean and pass it; this improves the GP's noise model.

A helper function that loads a list of previous results is already implemented in `degradation_optimizer.py`:

```
from recommenders.degradation_optimizer import load_previous_data
load_previous_data(ax_client, all_results) # all_results is a list of dicts
```

8.3 Fixing the random seed

For reproducibility, set `seed` in the Sobol step's `model_kwargs` (as shown above). The Bayesian steps are deterministic given the same data; only the initialization is stochastic.

9. Acquisition Functions

The acquisition function (AF) decides *which* point to suggest next. This is the "brain" of the optimizer after the GP model is fit.

9.1 Single-objective options

AF	Code	When to use
----	------	-------------

Log Expected Improvement (qLogEI)	<code>from botorch.acquisition.logei import qLogExpectedImprovement</code>	Default for single-objective. More numerically stable than qEI at low noise. Recommended.
Expected Improvement (qEI)	<code>from botorch.acquisition.monte_carlo import qExpectedImprovement</code>	Classic; slightly less stable than qLogEI but equivalent in practice.
Upper Confidence Bound (UCB)	<code>Models.GPEI</code> via <code>GPEI</code> backend	High-level abstraction; uses UCB-like logic. Good for quick setup.
Thompson Sampling	custom	Low overhead; good for parallel/async settings. Rarely needed at this scale.
GPEI (Ax high-level)	<code>Models.GPEI</code>	Ax's built-in single-objective GP+EI. Equivalent to qEI but less configurable.

For single-objective work, use `Models.GPEI` (simplest) or inject `qLogExpectedImprovement` via `BOTORCH_MODULAR` (more control):

```
from botorch.acquisition.logei import qLogExpectedImprovement
GenerationStep(
    model=Models.BOTORCH_MODULAR, model_gen_kwargs={"botorch_acqf_class":
qLogExpectedImprovement, "deduplicate": True}, )
```

9.2 Multi-objective options

AF	Code	Notes
qNEHVI	<code>qNoisyExpectedHypervolumeImprovement</code>	Recommended for MOO. Maximizes the hypervolume improvement of the Pareto frontier relative to noisy observations. State-of-the-art.

EHVI	(older, non-noisy version)	Assumes noiseless observations; brittle in practice. Avoid.
MOO (Ax high-level)	<code>Models.MOO</code>	Ax's wrapper around qNEHVI. Slightly less configurable but easier to set up.
ParEGO	not directly in AxClient	Scalarizes objectives randomly each step; simpler but weaker for 3+ objectives.

Your problem has 3 objectives (maximize degradation ratio, maximize rate constant, minimize acid excess). Use **qNEHVI** via `BOTORCH_MODULAR` — this is exactly what `degradation_optimizer.py` does.

9.3 How acquisition functions trade off exploration vs. exploitation

All EI-family functions balance: - **Exploitation**: suggest the point the model predicts will be best. - **Exploration**: suggest points the model is most uncertain about.

You cannot directly tune this balance in standard qNEHVI/qLogEI. If you find the optimizer converging too fast to a local optimum, increase the number of Sobol trials. If it is exploring too broadly and never converging, the model may need more data — increase the Sobol budget and/or batch size.

10. Reporting Results Back

```
ax_client.complete_trial( trial_index=trial_index, raw_data={ "max_degradation_ratio":
(value, sem), "degradation_rate_constant": (value, sem), "acid_molar_excess": (value,
sem), # echo the input back }, )
```

If the experiment fails (equipment error, no product detected), mark the trial as failed instead of reporting bad data:

```
ax_client.log_trial_failure(trial_index=trial_index)
```

Failed trials are excluded from the model. Do not report artificial zeros for failed experiments — this corrupts the GP.

11. Reading the Outputs

11.1 Pareto frontier (multi-objective)

```
pareto = ax_client.get_pareto_optimal_parameters() # Returns dict: {trial_index:
(parameters_dict, (means_dict, covariances_dict))} for trial_idx, (params, (means, _)) in
pareto.items(): print(f"Trial {trial_idx}: {params}") print(f" -> degradation ratio:
{means['max_degradation_ratio']:.3f}") print(f" -> rate constant:
{means['degradation_rate_constant']:.3f}") print(f" -> acid excess:
{means['acid_molar_excess']:.2f}")
```

The Pareto frontier is a set of *non-dominated* solutions: no point in the set is simultaneously better than another on all objectives. You must choose which trade-off is acceptable based on domain judgment.

11.2 Best single trial (single-objective or for reference)

```
best_params, best_values, trial_index = ax_client.get_best_parameters()
```

11.3 Exporting all trials as a DataFrame

```
df = ax_client.get_trials_data_frame()
```

This contains all trial parameters, outcomes, and trial statuses. Useful for post-hoc analysis or plotting the optimization trajectory.

12. Full Example for Your Problem

The complete implementation is in `recommenders/degradation_optimizer.py`.

A minimal usage example:

```
from recommenders.degradation_optimizer import create_model, get_suggestions, add_result
ax_client = create_model( seed=42, num_initial_recs=10, bayesian_batch_size=1,
acid_choices=["TFA", "HCl", "H2SO4"], solvent_choices=["toluene", "2-MeTHF", "EtOAc",
"DCM"], acid_molar_excess_bounds=(1.0, 5.0), ) # Optional: load previous data # from
recommenders.degradation_optimizer import load_previous_data #
load_previous_data(ax_client, previous_results_list) for round_num in range(20):
suggestions = get_suggestions(ax_client, n=1) for params, trial_index in suggestions:
print(f"Run: acid={params['acid']}, solvent={params['solvent']}, "
f"excess={params['acid_molar_excess']:.2f}") # --- run your experiment here --- ratio,
```

```
rate, excess = run_experiment(params) # -----
add_result(ax_client, trial_index, { "max_degradation_ratio": ratio,
"degradation_rate_constant": rate, "acid_molar_excess": excess, }) pareto =
ax_client.get_pareto_optimal_parameters()
```

13. Practical Tips

Batch size: Run 1 trial at a time if possible. Running batches (2–4 trials simultaneously) is more efficient for time but requires Ax to use Monte Carlo approximations that are slightly less accurate.

Scale your inputs: Ax handles internal normalization, but if your range parameter spans many orders of magnitude (e.g. 0.001–10 molar), set `"log_scale": True` in the parameter definition.

Don't report bad zeros: If an experiment yields no detectable product, either log it as a true 0.0 if that reflects reality, or use `log_trial_failure` if the measurement itself is invalid.

Track the Sobol → Bayesian transition: Ax logs which `GenerationNode` produced each trial. In `get_trials_data_frame()`, look for trials generated by "Sobol" vs "BoTorch" to understand when exploitation started.

Save the experiment state: Use `ax_client.save_to_json_file("experiment.json")` and `ax_client.load_from_json_file("experiment.json")` to persist across sessions.

Reproducibility: Fix the Sobol seed in `model_kwargs={"seed": 42}`. The Bayesian steps are deterministic conditional on data.

14. When to End the Campaign

Yes, this should be explicit. In practice, most campaigns should use a combination of hard limits and early-stop rules.

Recommended stopping criteria: - **Trial budget reached:** stop after N total completed trials. -

Material/time budget reached: stop when reagent volume, instrument time, or wall clock limit is hit. -

Plateau rule: stop if no meaningful improvement in best utility for k consecutive Bayesian trials (for example, <2-5% improvement over 8-12 trials). - **Target-achieved rule:** stop early if you have at least one solution that meets your acceptance thresholds (for example: ratio >= target, rate >= target, acid <= target). - **Pareto stability rule (MOO):** stop if the Pareto frontier changes only minimally for several rounds.

Practical default for your use case: - Set a fixed budget first (for example 20-40 trials). - Add a target-achieved early stop (scientific success criterion). - Add a plateau guard so you do not spend the

final budget on flat improvements.

Minimal threshold-based early-stop check:

```
def success(metrics): return ( metrics["max_degradation_ratio"] >= 2.5 and  
metrics["degradation_rate_constant"] >= -0.1 and metrics["acid_molar_excess"] <= 2.0 ) #  
In your optimization loop, stop if any Pareto point satisfies success(...)
```

For multi-objective campaigns, avoid a single "best" score as the only stop signal. Instead, define what "good enough" means scientifically, then stop once the frontier contains at least one decision-ready condition.

15. Further Reading

- [Ax tutorials \(v1.2\)](#) — new API, but the conceptual explanation is accurate for v1.1 too
- [BoTorch qNEHVI paper](#) — the MOO acquisition function used here
- [A Tutorial on Bayesian Optimization](#) — accessible overview (Frazier, 2018)
- `recommenders/degradation_optimizer.py` in this repo — the reference implementation for your exact problem setup